

Lista de Exercícios: Assembly MIPS

1) Implemente a seguinte função C em assembly MIPS. Siga as convenções de chamada para passagem de argumentos e retorno de valor.

```
1 int check(int a, int b) {  
2     if (a == b) {  
3         return 1;  
4     } else {  
5         return 0;  
6     }  
7 }
```

2) Traduza o código C a seguir para MIPS. Suponha que `sum` esteja em `$s0`, `i` em `$t0`, `len` em `$s1`, e o endereço base de `V` em `$s2`.

```
1 sum = 0;  
2 i = 0;  
3 while (i < len) {  
4     sum += V[i];  
5     i++;  
6 }
```

3) Considere sua implementação MIPS do 2). Se `len` for inicializado com 8:

- a) Quantas instruções MIPS **estáticas** sua implementação possui?
- b) Qual é o número total de instruções MIPS que são **executadas** (dinâmicas)?

4) Para as instruções MIPS a seguir, qual é a instrução C correspondente? Suponha que as variáveis `g`, `h`, e `i` estejam nos registradores `$s0`, `$s1`, e `$s2`, e o endereço de base do array `A` esteja em `$s6`.

```
1 sll $t0, $s2, 2  
2 add $t0, $s6, $t0  
3 lw $t1, 0($t0)  
4 sub $s0, $s1, $t1
```

5) Traduza a seguinte linha de código C para assembly MIPS. Suponha que `i` e `j` estão em `$s0` e `$s1`, e os endereços base de `A` e `B` estão em `$s2` e `$s3`.

```
1 A[i] = B[j - 2] + 5;
```

6) Traduza o loop MIPS a seguir para C. Suponha que *i* está em *\$t1*, *result* em *\$s2*, e *\$s0* mantém o endereço de base do array *MemArray*.

```

1      li    $t1, 0
2  LOOP:
3      lw    $s1, 0($s0)
4      add   $s2, $s2, $s1
5      addi  $s0, $s0, 4
6      addi  $t1, $t1, 1
7      slti  $t2, $t1, 100
8      bne   $t2, $zero, LOOP

```

7) Traduza o loop MIPS a seguir para C. Suponha que *i* está em *\$s1*, *n* em *\$s2*, e *\$s0* mantém o endereço base do array *Valores*.

```

1  move $s1, $s2
2  LOOP:
3      blez $s1, END_LOOP
4      sll  $t0, $s1, 2
5      add  $t1, $s0, $t0
6      lw   $t2, 0($t1)
7      sll  $t2, $t2, 1
8      sw   $t2, 0($t1)
9      addi $s1, $s1, -1
10     j     LOOP
11  END_LOOP:

```

8) Traduza a seguinte linha de código C para MIPS. Suponha que *i* e *j* estão em *\$s0* e *\$s1*, e os endereços base de *A*, *B*, e *D* em *\$s2*, *\$s3*, e *\$s4*.

```

1  D[i] = A[i] + B[j];

```

9) Para o código MIPS a seguir, qual é a instrução C correspondente? Suponha que *g*, *h*, e *i* estejam em *\$s0*, *\$s1*, e *\$s2*, e o endereço base de *A* em *\$s6*.

```

1  addi $t0, $s2, 4
2  sll  $t0, $t0, 2
3  add  $t0, $s6, $t0
4  lw   $t1, 0($t0)
5  sub  $s0, $t1, $s1

```

10) Traduza o loop MIPS a seguir para um laço *while* ou *do-while* em C. Suponha que *i* está no registrador *\$s1* e o endereço base do array *A* está em *\$s0*.

```

1      addi $s1, $zero, -1
2  LOOP:
3      addi $s1, $s1, 1
4      sll  $t0, $s1, 2
5      add  $t1, $s0, $t0
6      lw   $t2, 0($t1)
7      beq  $t2, $zero, LOOP

```

11) Traduza o laço for a seguir para MIPS. Suponha que *i* e *n* estão nos registradores \$s0 e \$s1, e os endereços de base de A e B em \$s2 e \$s3.

```

1  for (i = 0; i < n; i++) {
2      B[i] = A[n - 1 - i];
3  }

```

12) Implemente a função `processa_soma` em MIPS. Esta é uma função **não-folha**. Explique detalhadamente quais registradores precisam ser salvos na pilha e por quê. Suponha que a função `soma_dois(int x, int y)` já existe.

```

1  int processa_soma(int a, int b, int c, int d) {
2      int temp1 = soma_dois(a, b);
3      int temp2 = soma_dois(c, d);
4      return temp1 - temp2;
5  }

```

Soluções

1)

```
1 check:
2     bne $a0, $a1, else_case
3 if_case:
4     li $v0, 1
5     j exit_check
6 else_case:
7     li $v0, 0
8 exit_check:
9     jr $ra
```

2)

```
1
2     li $s0, 0
3     li $t0, 0
4 Loop:
5
6     bge $t0, $s1, EndLoop
7
8
9     sll $t2, $t0, 2
10    add $t2, $t2, $s2
11    lw $t3, 0($t2)
12    add $s0, $s0, $t3
13
14
15    addi $t0, $t0, 1
16    j Loop
17 EndLoop:
```

3) a) **Instruções estáticas:** A implementação possui **8 instruções** ('li', 'li', 'bge', 'sll', 'add', 'lw', 'add', 'addi', 'j'). A inicialização conta. Se contar apenas o laço, são 7. b) **Instruções dinâmicas:** O laço executa para $i = 0..7$ (8 vezes). A verificação final ($i=8$) falha. (2 instruções de init) + (8 iterações $\times$ 7 inst. no laço) + (1 inst. 'bge' final) = 2 + 56 + 1 = **59 instruções**.

4)

```
1 g = h - A[i];
```

5)

```
1 addi $t0, $s1, -2
2 sll $t0, $t0, 2
3 add $t0, $t0, $s3
4 lw $t1, 0($t0)
5 addi $t1, $t1, 5
6 sll $t2, $s0, 2
7 add $t2, $t2, $s2
8 sw $t1, 0($t2)
```

6)

```
1 // Assumindo que 'result' e 'MemArray' foram inicializados previamente.
2 int i;
3 for (i = 0; i < 100; i++) {
4     result += MemArray[i];
5 }
```

7)

```
1 // Assumindo que 'n', 'i', e 'Valores' foram inicializados previamente.
2 for (i = n; i > 0; i--) {
3     Valores[i] = Valores[i] * 2;
4 }
```

8)

```
1 sll $t0, $s0, 2
2 add $t0, $t0, $s2
3 lw $t1, 0($t0)
4 sll $t2, $s1, 2
5 add $t2, $t2, $s3
6 lw $t3, 0($t2)
7 add $t1, $t1, $t3
8 sll $t0, $s0, 2
9 add $t0, $t0, $s4
10 sw $t1, 0($t0)
```

9)

```
1 g = A[i + 4] - h;
```

10)

```
1 i = -1;
2 do {
3     i++;
4 } while (A[i] == 0);
```

11)

```

1      li    $s0, 0
2 Loop:
3      bge   $s0, $s1, EndLoop
4      sub   $t0, $s1, $s0
5      addi  $t0, $t0, -1
6      sll   $t1, $t0, 2
7      add   $t1, $t1, $s2
8      lw    $t2, 0($t1)
9      sll   $t3, $s0, 2
10     add   $t3, $t3, $s3
11     sw    $t2, 0($t3)
12     addi  $s0, $s0, 1
13     j     Loop
14 EndLoop:

```

12) Análise sobre Pilha e Registradores: A função `processa_soma` é uma função **não-folha** porque chama outra função (`soma_dois`), o que exige o salvamento de estado na pilha.

1. Registradores que PRECISAMOS salvar:

- `$ra` (Endereço de Retorno): A instrução `jal` sempre sobrescreverá `$ra`. Se não salvarmos o endereço de retorno de `processa_soma`, ela não saberá como voltar para seu chamador.
- Argumentos e Resultados Intermediários: Os argumentos `c` e `d` (`$a2`, `$a3`) precisam ser salvos antes da primeira chamada `jal`, pois os registradores `$a0`–`$a3` serão usados para passar argumentos. O resultado da primeira chamada, `temp1`, também precisa ser salvo antes da segunda chamada.

2. Registradores que NÃO PRECISAMOS salvar:

- `$s0`–`$s7`: Como a função não usa nenhuma variável de longa duração que precise persistir entre as chamadas, não precisamos salvar/restaurar nenhum registrador `$s`.

```

1 processa_soma:
2     # --- Prologo ---
3     # Aloca 4 words (16 bytes) na pilha para:
4     # 1. $ra (endereço de retorno)
5     # 2. $a2 (argumento 'c')
6     # 3. $a3 (argumento 'd')
7     # 4. temp1 (resultado da primeira soma)
8     addi  $sp, $sp, -16
9     sw    $ra, 12($sp)
10    sw    $a2, 8($sp)
11    sw    $a3, 4($sp)
12
13    # --- Primeira chamada: soma_dois(a, b) ---
14    # a e b ja estao em $a0, $a1
15    jal   soma_dois
16    # Salva o resultado (temp1) na pilha

```

```

17      sw    $v0, 0($sp)
18
19      # --- Segunda chamada: soma_dois(c, d) ---
20      # Carrega 'c' e 'd' da pilha para os registradores de argumento
21      lw     $a0, 8($sp)
22      lw     $a1, 4($sp)
23      jal    soma_dois
24      # Agora $v0 contem o resultado de temp2
25
26      # --- Calculo final: temp1 - temp2 ---
27      lw     $t0, 0($sp)      # Carrega temp1 salvo
28      sub    $v0, $t0, $v0    # $v0 = temp1 - temp2
29
30      # --- Epilogo ---
31      # Restaura os valores e libera a pilha
32      lw     $ra, 12($sp)
33      addi   $sp, $sp, 16
34      jr     $ra

```